Design and Analysis of Algorithm

Assignment: Lab 01

Submitted To:
Dr. Ajaya Kumar Dash

Submitted By:
Soumya Ranjan Behera
ID: B125124
CSE-B-2

Academic Year: 2026-2027

3rd Semester

Question 1:

Put them in Order: Using implementation in **C**, place the given functions in a list by *increasing order of growth* for sufficiently large values of n.

$n \log_2 n$	$12\sqrt{n}$	$\frac{1}{n}$	$n^{\log_2 n}$
$100n^2 + 6n$	$n^{0.51}$	$n^2 - 324$	$50n^{0.5}$
$2n^3$	3^n	$2^{32}n$	$\log_2 n$

Source Code (q1.c)

```
#include <stdio.h>
#include <stdlib.h>

// Struct to hold function properties for comparison
typedef struct {
    const char *name;
    double exp_base;
    double log2_pow;
    double n_pow;
    double log_pow;
    double constant;
} FunctionClass;

// Comparison function
int compare(const void *a, const void *b) {
    FunctionClass *A = (FunctionClass *)a;
    FunctionClass *B = (FunctionClass *)b;

    if (A->exp_base != B->exp_base) return (A->exp_base < B->exp_base) ? -1 :
1;
    if (A->log2_pow != B->log2_pow) return (A->log2_pow < B->log2_pow) ? -1 :
1;
    if (A->n_pow != B->n_pow)       return (A->n_pow < B->n_pow) ? -1 : 1;
    if (A->log_pow != B->log_pow)   return (A->log_pow < B->log_pow) ? -1 : 1;
    if (A->constant != B->constant) return (A->constant < B->constant) ? -1 :
1;

    return 0;
}

int main() {
    // Define the functions to be compared
    FunctionClass items[] = {
        {"1 / n",          1, 0, -1,   0, 1},
        {"log2(n)",        1, 0, 0,    1, 1},
        {"12 * sqrt(n)",   1, 0, 0.5,  0, 12},
        {"50 * n^0.5",     1, 0, 0.5,  0, 50},
        {"n^0.51",         1, 0, 0.51, 0, 1},
        {"2^32 * n",       1, 0, 1,    0, 4294967296.0},
        {"n * log2(n)",    1, 0, 1,    1, 1},
        {"n^2 - 324",      1, 0, 2,    0, 1},
```

```
        {"100n^2 + 6n",    1, 0, 2,    0, 100},
        {"2n^3",           1, 0, 3,    0, 2},
        {"n^(log2 n)",     1, 1, 0,    0, 1},
        {"3^n",            3, 0, 0,    0, 1}
    };

    int num_items = sizeof(items) / sizeof(items[0]);

    qsort(items, num_items, sizeof(FunctionClass), compare);

    printf("Functions strictly ordered by increasing asymptotic growth
rate:\n");
    for (int i = 0; i < num_items; i++) {
        printf("%2d. %-20s\n", i + 1, items[i].name);
    }

    FILE *gp = popen("gnuplot", "w");
    if (gp == NULL) {
        perror("Error opening gnuplot. Make sure gnuplot is installed.");
        return 1;
    }

    fprintf(gp, "set terminal png size 1000,800\n");
    fprintf(gp, "set output 'growth_rates.png'\n");

    // Scale and ranges to prevent math overflows
    fprintf(gp, "set logscale xy\n");
    fprintf(gp, "set xrange [1:1e12]\n");
    fprintf(gp, "set yrange [1e-13:1e200]\n");

    // Set labels and title
    fprintf(gp, "set title 'Asymptotic Growth Rates of Functions'\n");
    fprintf(gp, "set xlabel 'n'\n");
    fprintf(gp, "set ylabel 'f(n)'\n");
    fprintf(gp, "set key spacing 1.5\n");
    fprintf(gp, "set key outside right center\n");

    // Plot command using gnuplot's default styles/colors
    fprintf(gp, "plot "
                "x*(log(x)/log(2)) title 'n log2(n)', "
                "12*x**(0.5) title '12 * n^{0.5}', "
                "1.0/x title '1/n', "
                "(x <= 1e6 ? x**(log(x)/log(2)) : 1/0) title 'n^{log2 n}', "
                "100*x**2 + 6*x title '100n^2 + 6n', "
                "x**(0.51) title 'n^{0.51}', "
                "(x > 18 ? x**2 - 324 : 1/0) title 'n^2 - 324', "
                "50*x**(0.50) title '50 * n^{0.5}', "
                "2*x**3 title '2n^3', "
                "(x <= 415 ? 3**x : 1/0) title '3^n', "
                "x*4294967296.0 title '2^{32} * n', "
                "(log(x)/log(2)) title 'log2(n)' \n");

    fprintf(gp, "exit\n");
    pclose(gp);
```

```
    return 0;
}
```

Complexity Summary

Time Complexity: O(N log N) where N is the number of functions being sorted.
Space Complexity: O(N) to store the function properties array.

Observations

1. Terminal Output:

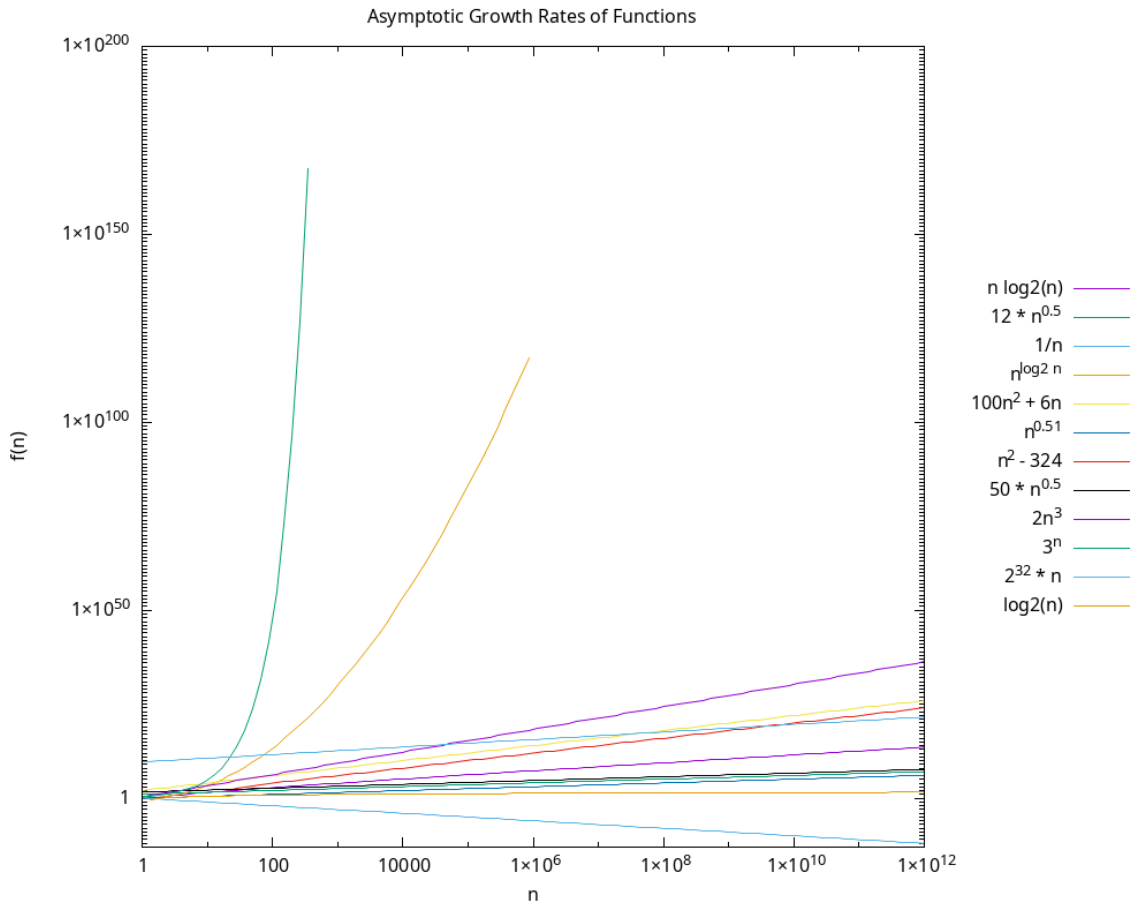
Functions strictly ordered by increasing asymptotic growth rate:

```
 1. 1 / n
 2. log2(n)
 3. 12 * sqrt(n)
 4. 50 * n^0.5
 5. n^0.51
 6. n * log2(n)
 7. 2^32 * n
 8. n^2 - 324
 9. 100n^2 + 6n
10. 2n^3
11. n^(log2 n)
12. 3^n
```

Why the Graph Looks the Way It Does

Using a Log Scale: Both the x and y-axes use a logarithmic scale (set logscale xy). This is absolutely necessary to balance out the wild extremes of these functions. Without a log scale, there's no way you could fit a function that shrinks down to almost zero (like 1/n dropping to 10^{-13}) on the same standard chart as a function that explodes upwards (like 3^n hitting 10^{200}).

Output Visualization



Question 2:

Fair vs Biased coin: Using simulation in C, show that the probability of getting a *HEAD* by tossing a *fair coin* is about 0.5. Extend your simulation to compare fair vs biased coin-tossing experiments.

Understanding the Law of Large Numbers: A Coin Toss Simulation

The Law of Large Numbers is a fundamental theorem in probability theory that describes the result of performing the same experiment a large number of times. According to this law, the experimental probability will converge to the expected theoretical probability as the number of trials increases.

Source Code (q2.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

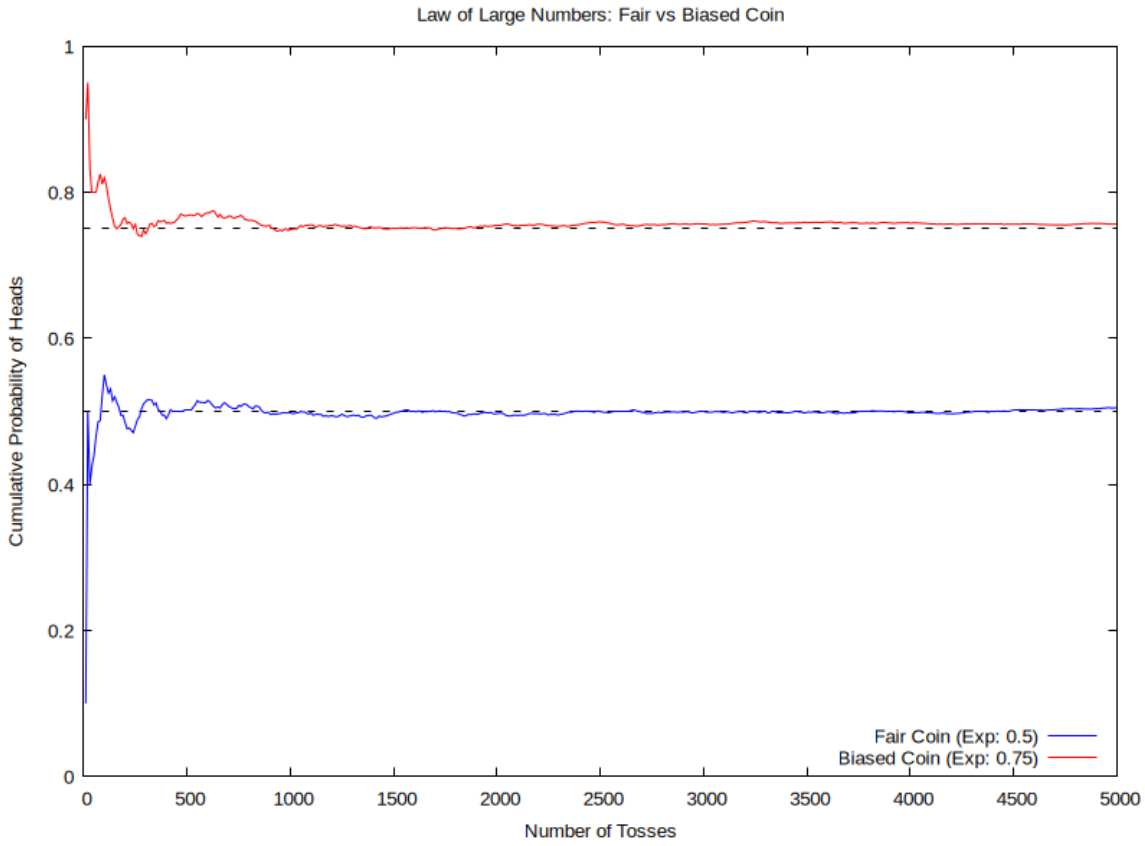
```
// Helper function for a Fair Coin toss
int tossFair() {
    return rand() % 2;
}

// Helper function for a Biased Coin toss
int tossBiased(double bias) {
    double r = (double)rand() / RAND_MAX;
    return (r < bias) ? 1 : 0;
}

int main() {
    // Seed the random number generator
    srand((unsigned int)time(NULL));

    int max_tosses = 5000; // Total number of tosses for the simulation
    double bias_prob = 0.75; // 75% chance of Heads for the biased coin

    int fair_heads = 0;
    int biased_heads = 0;

    // Arrays to store cumulative probability values for plotting
    // Sized to max_tosses + 1 to avoid out-of-bounds errors
    double fair_probs[max_tosses + 1];
    double biased_probs[max_tosses + 1];

    // Simulation loop
    for (int i = 1; i <= max_tosses; i++) {
        fair_heads += tossFair();
        biased_heads += tossBiased(bias_prob);

        // Calculate the cumulative probability of Heads up to the i-th toss
        fair_probs[i] = (double)fair_heads / i;
        biased_probs[i] = (double)biased_heads / i;
    }

    // Output final results to console to explicitly show the probabilities
    printf("--- Simulation Complete (%d tosses) ---\n", max_tosses);
    printf("Expected Fair Coin Probability:   0.500000\n");
    printf("Calculated Fair Coin Probability: %f\n\n", fair_probs[max_tosses]);
    printf("Expected Biased Coin Probability: %f\n", bias_prob);
    printf("Calculated Biased Coin Prob:      %f\n", biased_probs[max_tosses]);

    // --- Visualization using Gnuplot ---
    FILE *gnuplot = popen("gnuplot -persistent", "w");
    if (gnuplot == NULL) {
        printf("Error: Could not open Gnuplot.\n");
        return 1;
    }

    // Graph setup
    fprintf(gnuplot, "set terminal pngcairo enhanced font 'arial,10' size
800,600\n");
    fprintf(gnuplot, "set output 'coin_simulation.png'\n");
    fprintf(gnuplot, "set title 'Law of Large Numbers: Fair vs Biased
```

```
Coin'\n");
    fprintf(gnuplot, "set xlabel 'Number of Tosses'\n");
    fprintf(gnuplot, "set ylabel 'Cumulative Probability of Heads'\n");
    fprintf(gnuplot, "set yrange [0:1]\n");
    fprintf(gnuplot, "set key right bottom\n"); // Move legend so it doesn't
block data

    // Add horizontal reference lines at Expected Probabilities (0.5 and 0.75)
    fprintf(gnuplot, "set arrow from 0,0.5 to %d,0.5 nohead lc rgb 'black'
dashtype 2\n", max_tosses);
    fprintf(gnuplot, "set arrow from 0,0.75 to %d,0.75 nohead lc rgb 'black'
dashtype 2\n", max_tosses);

    // Plotting as lines to show convergence
    fprintf(gnuplot, "plot '-' title 'Fair Coin (Exp: 0.5)' with lines lc rgb
'blue', \\\n");
    fprintf(gnuplot, "     '-' title 'Biased Coin (Exp: 0.75)' with lines lc
rgb 'red'\n");

    // We plot every 10th step to make the graph render cleanly
    int step = 10;

    // Send Fair Coin Data
    for (int i = step; i <= max_tosses; i += step) {
        fprintf(gnuplot, "%d %f\n", i, fair_probs[i]);
    }
    fprintf(gnuplot, "e\n");

    // Send Biased Coin Data
    for (int i = step; i <= max_tosses; i += step) {
        fprintf(gnuplot, "%d %f\n", i, biased_probs[i]);
    }
    fprintf(gnuplot, "e\n");

    pclose(gnuplot);
    return 0;
}
```

Complexity Summary

Time Complexity: O(N) where N is the total number of coin tosses (5000).
Space Complexity: O(N) for storing the cumulative probability arrays for plotting.

Output Visualization



Observations

If we examine the visualization provided in the file "coin_simulation.png", we will notice the pattern that perfectly illustrates the theorem.

- **Initial Variance:** In the early stages of the experiment (roughly between 0 and 500 tosses), the cumulative probability lines experience sharp spikes and deep valleys. We can see that initially the probability was fluctuating significantly because the sample size is small—meaning every single toss has a heavy mathematical impact on the running average.
- **Long-Term Convergence:** Notice how the probability later fixed as we grew the number of cases. As the number of tosses steadily climbs toward 5000, the blue line (fair coin) tightly hugs the expected 0.5 mark, while the red line (biased coin) seamlessly stabilizes at the 0.75 mark.

Conclusion

This visual representation shows exactly why large sample sizes are required for reliable statistical data. While a small number of trials leaves room for high variance and unpredictable fluctuations, increasing the number of cases smooths out the anomalies, converging the outcome onto the true theoretical probability.

Question 3:

Performance analysis of bubble sort: Using C, implement two different versions of bubble sort simulation for randomised data sequences as follows:

(*i*) Bubble sort that terminates if the array is sorted before the $(n-1)^{th}$ pass.

(*ii*) Bubble sort that always completes the $(n-1)^{th}$ pass.

Plot the number of comparisons in both cases to analyse their efficiency.

Bubble Sort Performance Analysis: Standard vs. Optimized

This document outlines the implementation and performance comparison between standard and optimized Bubble Sort algorithms, specifically analyzing the number of comparisons made as the array size increases.

Implementations

We have implemented two variants of the Bubble Sort algorithm in C:

1. **Standard Bubble Sort (Unoptimized)**:

This implementation always completes the `(n-1)` passes, regardless of whether the array is already sorted during the process. It compares adjacent elements and swaps them if they are in the wrong order.

2. **Optimized Bubble Sort (Early Termination)**:

This variant introduces a `swapped` boolean flag. During each pass, if no elements are swapped, it means the array is already sorted. The algorithm then terminates early, avoiding unnecessary subsequent passes.

Array Composition

For our trials, we constructed a specialized array where:

- The **first half** of the array consists of completely randomized numbers.
- The **second half** consists of strictly increasing, already sorted numbers.

Why avoid a purely randomized array

- In large random arrays, the chance of elements naturally sorting at the end is near zero, meaning optimized bubble sort rarely stops early. The performance difference between the standard and optimized versions becomes negligible. Instead, using an array that is half-random and half-sorted guarantees early termination, clearly demonstrating the optimized algorithm's efficiency advantage.

Source Code (q3.c)

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <stdbool.h>
#include <time.h>

// Standard Bubble Sort: Always completes the (n-1)th pass
void bubbleSortStandard(int arr[], int n, long long *comparisons) {
    *comparisons = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            (*comparisons)++;
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Optimized Bubble Sort: Terminates if array is sorted before (n-1)th pass
void bubbleSortOptimized(int arr[], int n, long long *comparisons) {
    *comparisons = 0;
    bool swapped;
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            (*comparisons)++;
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) {
            break;
        }
    }
}

int main() {
    int min_size = 100;
    int max_size = 2000;
    int step = 100;
    int trials = 5;

    int num_points = (max_size - min_size) / step + 1;

    int *sizes = (int *)malloc(num_points * sizeof(int));
    long long *std_data = (long long *)malloc(num_points * sizeof(long long));
    long long *opt_data = (long long *)malloc(num_points * sizeof(long long));

    srand((unsigned int)time(NULL));
    printf("Running simulations and buffering data...\n");

    // 1. Run simulations and store results in arrays
```

```
    int data_index = 0;
    for (int n = min_size; n <= max_size; n += step) {
        long long totalCompStd = 0;
        long long totalCompOpt = 0;

        for (int t = 0; t < trials; t++) {
            int *arrStd = (int *)malloc(n * sizeof(int));
            int *arrOpt = (int *)malloc(n * sizeof(int));

            // Populate the first half with randomized data
            int split_index = n / 2;
            for (int i = 0; i < split_index; i++) {
                int val = rand() % 10000;
                arrStd[i] = val;
                arrOpt[i] = val;
            }

            // Populate the second half with strictly increasing, sorted data
            int current_sorted_val = 10000;
            for (int i = split_index; i < n; i++) {
                current_sorted_val += (rand() % 10) + 1;
                arrStd[i] = current_sorted_val;
                arrOpt[i] = current_sorted_val;
            }

            long long compStd = 0, compOpt = 0;
            bubbleSortStandard(arrStd, n, &compStd);
            bubbleSortOptimized(arrOpt, n, &compOpt);

            totalCompStd += compStd;
            totalCompOpt += compOpt;

            free(arrStd);
            free(arrOpt);
        }

        sizes[data_index] = n;
        std_data[data_index] = totalCompStd / trials;
        opt_data[data_index] = totalCompOpt / trials;
        data_index++;
    }

    printf("Simulation complete. Piping data directly to gnuplot...\n");

    FILE *gnuplotPipe = popen("gnuplot -persistent", "w");
    if (gnuplotPipe) {
        fprintf(gnuplotPipe, "set terminal pngcairo size 800,600 enhanced font
'Arial,12'\n");
        fprintf(gnuplotPipe, "set output 'bubble_sort_performance.png'\n");
        fprintf(gnuplotPipe, "set title 'Performance Analysis: Bubble Sort
Comparisons'\n");
        fprintf(gnuplotPipe, "set xlabel 'Array Size (n)'\n");
        fprintf(gnuplotPipe, "set ylabel 'Number of Comparisons'\n");
        fprintf(gnuplotPipe, "set grid\n");
        fprintf(gnuplotPipe, "set key left top\n");
```

```
        fprintf(gnuplotPipe, "plot '-' using 1:2 with linespoints lw 2 pt 7
title 'Standard (Unoptimized)', \\\n");
        fprintf(gnuplotPipe, "     '-' using 1:2 with linespoints lw 2 pt 7
title 'Optimized (Early Termination)'\n");

        // Send Standard Bubble Sort data
        for (int i = 0; i < num_points; i++) {
            fprintf(gnuplotPipe, "%d %lld\n", sizes[i], std_data[i]);
        }
        fprintf(gnuplotPipe, "e\n");

        // Send Optimized Bubble Sort data
        for (int i = 0; i < num_points; i++) {
            fprintf(gnuplotPipe, "%d %lld\n", sizes[i], opt_data[i]);
        }
        fprintf(gnuplotPipe, "e\n");

        pclose(gnuplotPipe);
        printf("Success! Graph saved as 'bubble_sort_performance.png'.\n");
    } else {
        printf("Error: Could not open gnuplot. Is it installed?\n");
    }

    free(sizes);
    free(std_data);
    free(opt_data);

    return 0;
}
```

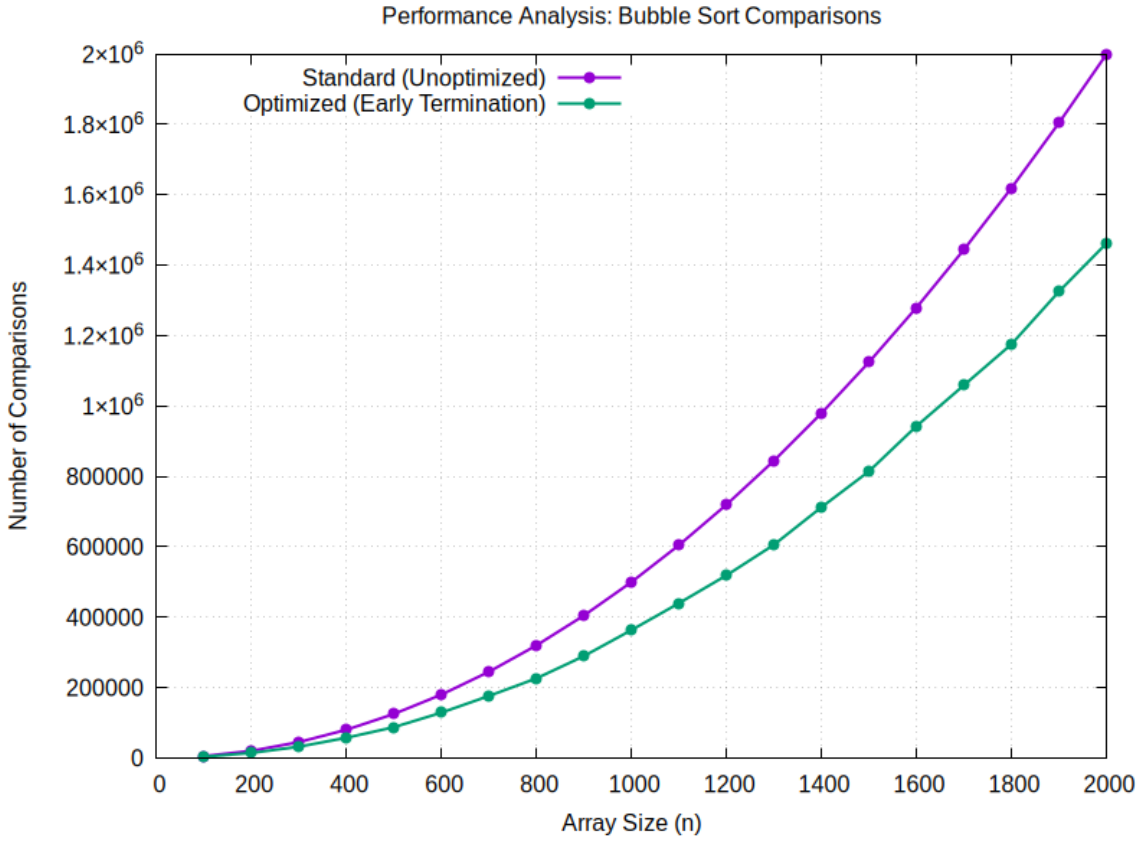
Complexity Summary

Time Complexity: $O(N^2)$ worst-case for both. O(N) best-case for the Optimized version if the array is already sorted.
Space Complexity: O(1) auxiliary space (in-place sorting).

Results

The visualization of the simulation clearly displays the divergence in the number of comparisons. The optimized bubble sort effectively avoids redundant comparisons in the sorted upper half of the array.

Output Visualization



Question 4:

Towers of Hanoi (ToH): Simulate the solution to the ToH problem using C. Plot the total number of moves required for solving the problem of n-discs. What can you conclude about your algorithm from the plot obtained?

Tower of Hanoi: Implementation and Complexity Analysis

This document outlines the implementation details of the provided C program, which calculates the minimum number of moves required to solve the Tower of Hanoi puzzle and visualizes the growth rate using Gnuplot.

Mathematical Complexity

The Tower of Hanoi problem is a classic example of exponential time complexity. To move `n` discs from the source peg to the destination peg, it takes exactly **$2^n - 1$** steps.

Complexity Summary

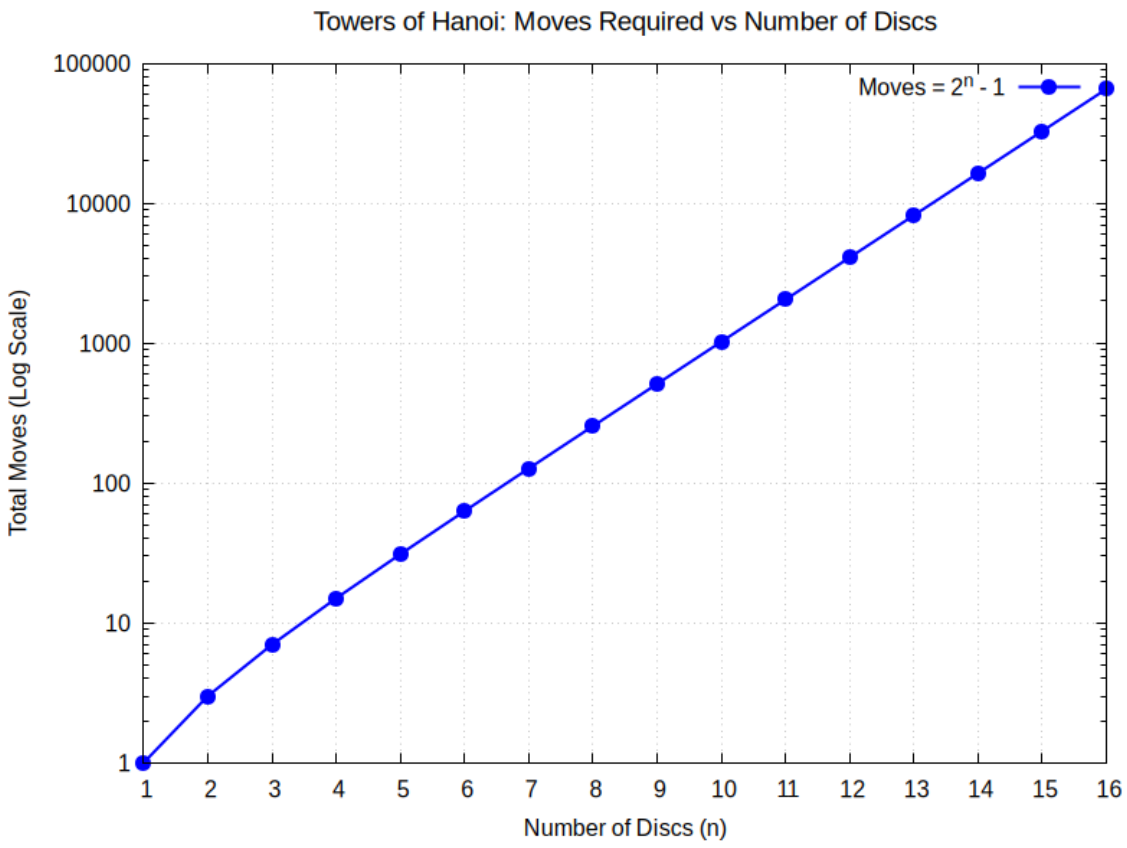
Time Complexity: $O(2^n)$ where n is the number of discs.
Space Complexity: O(n) auxiliary space for the recursion call stack.

Source Code (q4.c)

```
#include <stdio.h>
#include <stdlib.h>

// Recursive function to solve ToH and count moves
void towerOfHanoi(int n, char source, char aux, char dest, long long *moves) {
    if (n == 0) {
        return;
    }
    towerOfHanoi(n - 1, source, dest, aux, moves);
    (*moves)++;
    towerOfHanoi(n - 1, aux, source, dest, moves);
}

int main() {
    FILE *gnuplotPipe = popen("gnuplot -persistent", "w");

    if (gnuplotPipe == NULL) {
        printf("Error: Could not open Gnuplot. Ensure it is installed and in
your PATH.\n");
        return 1;
    }

    printf("Calculating ToH moves and generating plot...\n");

    // Configuring the plot
    fprintf(gnuplotPipe, "set terminal pngcairo size 800,600 enhanced font
'Arial,12'\n");
    fprintf(gnuplotPipe, "set output 'toh_moves.png'\n");
    fprintf(gnuplotPipe, "set title 'Towers of Hanoi: Moves Required vs Number
of Discs' font ',14' \n");
    fprintf(gnuplotPipe, "set xlabel 'Number of Discs (n)' font ',12' \n");
    fprintf(gnuplotPipe, "set ylabel 'Total Moves (Log Scale)' font ',12' \n");

    // Set logarithmic scale for the y-axis (base 10)
    fprintf(gnuplotPipe, "set logscale y 10\n");

    // Set x-axis range and ticks
    fprintf(gnuplotPipe, "set xrange [1:16]\n");
    fprintf(gnuplotPipe, "set xtics 1,1,16\n");
    fprintf(gnuplotPipe, "set grid\n");

    fprintf(gnuplotPipe, "plot '-' with linespoints linewidth 2 pointtype 7
pointsize 1.5 linecolor rgb 'blue' title 'Moves = 2^n - 1'\n");

    // Calculate moves for n = 1 to 16 and stream the data to Gnuplot
    for (int n = 1; n <= 16; n++) {
        long long moves = 0;
        towerOfHanoi(n, 'A', 'B', 'C', &moves);
        // printf("n = %d \t Moves = %lld\n", n, moves);
        fprintf(gnuplotPipe, "%d %lld\n", n, moves);
    }

    fprintf(gnuplotPipe, "e\n");
```

```
    pclose(gnuplotPipe);

    return 0;
}
```

Output Visualization



Question 5:

Find the partition point: Consider an array A with n elements containing a run of 0's followed by a run of 1's. Implement a method to find out the exact point of transition between them.

Implementation Details: Transition Point Search

This document outlines the key components and processes implemented in the provided C program.

1. Core Implementations

The program implements an efficient algorithm to find a specific target within a structured array:

- **Binary Search Algorithm (`findPartition`):** The code uses a modified binary search to find the index of the first `1` in a sorted array of `0`s and `1`s. It computes a `mid` point and checks if it is the first `1` (either it is at index 0 or the preceding element is `0`).
- **Traversal Logic:** If the `mid` element is `1` but not the first one, the search narrows to the left half (`high = mid - 1`). If the `mid` element is `0`, the search moves to the right half (`low = mid + 1`). If no `1` is found, the function returns `-1`.

Source Code (q5.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Function to find the transition point using Binary Search
int findPartition(int arr[], int n) {
    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Check if mid is the first 1
        if (arr[mid] == 1 && (mid == 0 || arr[mid - 1] == 0)) {
            return mid;
        }
        // If mid is 1, the first 1 must be to the left
        else if (arr[mid] == 1) {
            high = mid - 1;
        }
        // If mid is 0, the first 1 must be to the right
        else {
            low = mid + 1;
        }
    }
    // Return -1 if array is all 0s or empty
    return -1;
}

int main() {
    // 1. Define the size of the array
    int size_arr = 1000;
    int arr[size_arr];
    srand(time(NULL));

    // Pick a random transition index between 0 and size_arr
    int random_transition = rand() % (size_arr + 1);

    // Fill the array: 0s before the transition, 1s from the transition onward
    for (int i = 0; i < size_arr; i++) {
        if (i < random_transition) {
            arr[i] = 0;
        } else {
            arr[i] = 1;
        }
```

```
    }

    // Print the actual transition point we generated for comparison
    if (random_transition < size_arr) {
        printf("The actual random transition point is at index: %d\n",
random_transition);
    } else {
        printf("The array was filled completely with 0s.\n");
    }

    // 4. Test the algorithm on the randomly generated array
    int index = findPartition(arr, size_arr);

    if (index != -1)
        printf("-> Algorithm found partition point at index: %d\n", index);
    else
        printf("-> Algorithm found no partition point (no 1s in array).\n");

    return 0;
}
```

Complexity Summary

Time Complexity: O(log N) where N is the size of the array, due to Binary Search.
Space Complexity: O(1) auxiliary space as it is an iterative search.

Question 6:

Element uniqueness:: For given n random numbers, implement a method in C to check if there are any duplicates. What can you conclude about your method for a sufficiently large value of n?

Source Code (q6.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Function to count the total number of duplicate elements
int countDuplicates(int arr[], int n) {
    if (n <= 1) return 0;

    // Step 1: Find the maximum value in the array to appropriately size the
frequency array
    int max_val = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }

    // Step 2: Allocate a frequency array initialized to 0
```

```
    int *freq = (int *)calloc(max_val + 1, sizeof(int));
    if (freq == NULL) {
        return -1;
    }

    int duplicate_count = 0;

    // Step 3: Linearly scan the original array to count duplicates
    for (int i = 0; i < n; i++) {
        if (freq[arr[i]] > 0) {
            // If the frequency is already greater than 0, we have seen this
number before
            duplicate_count++;
        }
        freq[arr[i]]++; // Mark this number as seen
    }

    free(freq);

    return duplicate_count;
}

int main() {
    int size_arr, x;

    size_arr = 100;
    x = 1000; // x > size_arr so that we may have duplicates

    // Dynamically allocate memory based on size_arr
    int *arr = (int *)malloc(size_arr * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    srand(time(NULL));

    // Generate random values between 0 and x (inclusive)
    for (int i = 0; i < size_arr; i++) {
        arr[i] = rand() % (x + 1);
    }

    int total_duplicates = countDuplicates(arr, size_arr);

    printf("Result: Found %d duplicate(s) in the array.\n", total_duplicates);

    free(arr);

    return 0;
}
```

Complexity & Scalability

- **Time Complexity: O(N)** - Requires only two linear passes (finding the max value, and counting duplicates).

- **Space/Memory Complexity: O(M)** - Requires an auxiliary array of size **M + 1**, where **M** is the maximum value in the input array.
- **Conclusion for Large N:** For a sufficiently large value of **N**, this method remains highly performant and scales perfectly linearly in execution time. It avoids the severe bottlenecks of **O(N^2)** naive approaches and outperforms **O(N log N)** sorting methods, provided the maximum value **(M)** does not cause memory allocation issues.